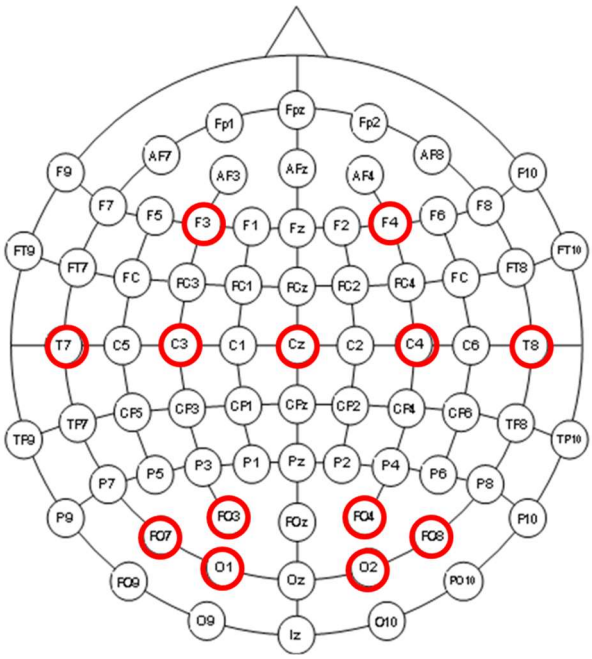
Advanced Techniques for EEG Processing - Laboratory Exercises

Exercise 6

The .mat file `REST_TASK_REST.mat` contains a set of 13 EEG signals (sampling rate = 128 Hz) acquired on a subject during an initial relax phase (R1), a central task phase (T) consisting in solving arithmetic operations, and a final relax phase (R2). Each phase lasts 5 minutes. The 13 electrodes used in the acquisition are indicated by red circles in the figure.



The .mat files contains the following quantities:

`X`: EEG data organized as a 2-D matrix (n x m where n is the number of electrodes =13 and m the number of time samples.
`ch_names`: a cell array containing the labels of the n electrodes.
`srate`: containing the value of the sampling rate.

The file `Standard-10-20-Cap13.locs` contains the location of the channels to obtain the topographical maps.

In the following points, plot the PSD (Power Spectral Density) in the range 0.5 Hz – 40 Hz

1) Load the .mat file and plot the n EEG signals as a function of time (over the entire duration and over 30 seconds, e.g. in the range 135 s-165 s).

2) Compute the PSD of the n electrodes over the entire duration=15 min (use `pwelch`, see page 2 for suggestions on parameters), and plot the PSD of all electrodes in a single figure (use `subplot)`.

3) Save the EEG data `X` in a .mat file (e.g. `dataExercise6.mat`)**..**

4) Obtain the demixing matrix and the topographic maps of the ICs using the toolbox EEGLAB, through the following steps.

4.a) Launch EEGLAB from the command window `>>eeglab`

4.b) Import in EEGLAB the saved .mat file and the file containing the channel locations. To this aim, from the EEGLAB GUI select '*File>Import Data>Using EEGLAB Functions and Plugins>From ASCII/float file or Matlab array*', then from the 'Browse' button on the top select the saved .mat file and in correspondence to '*Data sampling rate'* indicates the value of the sampling rate (in Hz). In the same window, use the 'Browse' button on the row of '*Channel location file or info*' to import the file 'Standard-10-20-Cap13.locs'. Leave unfilled the other edit boxes. Press OK and assign a name to the dataset. It is possible to display the time pattern of the imported signals '*Plot>Channel data (Scroll)*'.

4.c) Estimate the demixing matrix and the independent components of the imported dataset by selecting '*Tools>* D*ecompose Data by ICA*' and setting

ICA algorithm to use → *runica*

Commandline options → 'extended', 1, 'bias', 'off', 'rndreset', 'no'

Press Ok and wait for the algorithm to terminate. It is possible to display the time pattern of the estimated independent components selecting '*Plot>Component activations (Scroll)*'.

4.d) Display the topographic maps of the estimated independent component selecting '*Plot>Component maps>In 2-D*'. Save the (.fig) containing the topographic maps (e.g. `mapICs_Exercise6.fig`).

4.e) Export the demixing matrix selecting '*File>Export>Weight matrix to text file*', and give a name to the file specifying the extension .txt (e.g. `matrixW_Exercise6.txt`).

4.f) Save the current dataset created in EEGLAB selecting '*File>Save current dataset(s)*', assigning a name (two files are automatically saved with extension .set and .fdt).

4.g) Close the EEGLAB windows and clean the workspace (`clear`).

5) Return in your m. script and use function `load` to import the demixing matrix from the saved file. Use the demixing matrix to compute the estimated independent components and plot them as a function of time.

6) Compute the PSD of the n independent components over the entire duration (using `pwelch`) and plot the PSD of all ICs in a single figure (using `subplot`).

7) Based on the time pattern, topographical map, and PSD of the estimated ICs, identify the main artifact components. You can also use the function `plot_IC` (available in 'Virtuale', see page 3) to better inspect selected ICs (this function plots the time pattern, PSD and topographical map of an IC within a single figure).

8) Reconstruct the n EEG signals cleaned from the identified artifact components and compute the PSD of the cleaned signals over the entire duration. Plot the time pattern and plot the PSD of the clean EEG signals and compare them with those before artifact removal.

9) Using the cleaned EEG signals, compute the PSD of the 13 channels *separately* for the initial relax phase R1, for the central task phase T, and for the final relax phase R2 and plot them in three separate figures (one figure per phase).

10) For each phase separately, average the PSD over the frontal channels (F3 and F4), over the temporo-central channels (T7, C3, Cz, C4, T8), and over the parieto-occipital channels (PO3, PO4, PO7, PO8, O1, O2). In three subplots, plot the frontal PSD in each of the three phases (subplot1), the temporo-central PSD in each of the three phases (subplot2), the parieto-occipital PSD in each of the three phases (subplot3). Observe the differences among the phases, in particular in the alpha band (8-14 Hz).

11) Starting from the PSD computed for each scalp region (in point 10), compute the alpha power in each region (frontal, temporal-central, parietal-occipital) and for each phase. The alpha power is the area under the PSD curve between 8 and 14 Hz (use `trapz`, see page 3). In the same plot, display the value of alpha power in the three phases for each scalp region.

Function

```
[P,F]=pwelch(X,WINDOW,NOVERLAP,NFFT,Fs)
```

returns in `P` the power spectral density of `X`, estimated by dividing `X` into sections containing a number of samples specified in `WINDOW`. `NOVERLAP` specifies the samples of overlap between adjacent sections (leaving it at the default value [ ], a 50% overlap is used). `NFFT` is the number of points used to calculate the spectrum; if `NFFT` is greater than `WINDOW` the data in each section is zero-padded. `Fs` is the sampling frequency (= srate). `F` returns the vector of frequencies at which the power density is estimated (the resolution in Hz is equal to $Fs/NFFT$). Specifically, `F` is a $n_f \times 1$ vector of frequencies from 0 to $Fs/2$ with step $Fs/NFFT$ (thus $n_f = \frac{\text{NFFT}}{2} + 1$)

If `X` is a vector, it is converted to a column vector and treated as a single channel. If `X` is a matrix ($m \times n$), the power spectral density is calculated independently for each column and saved in the corresponding column of `P` (size of `P` = $n_f \times n$).

<u>Suggestion for computing PSD both of EEG signals and ICs</u>: use `WINDOW` of 5 seconds (`WINDOW=5*Fs`), NOVERLAP at the default value (`[]`), and use zero-padding to increase frequency resolution (`NFFT=2*WINDOW`). The frequency resolution is thus $F_s/NFFT = F_s/(10 \cdot F_s) = 0.1$ Hz.

Function (not built-in function)

```
plot_IC(index_ICs,A,Y,P_ICs,F,chan_locs,Fs,t1,t2)
```

for each selected IC (indicated in `index_ICs`, for example `index_ICs=[1 2 6])` generates a single figure containing the topographic map of the IC, the time pattern and the power spectral density. `A` is the estimated mixing matrix (= inv(W), where W is the demixing matrix obtained from EEGLAB); `Y` ($n \times m$) is the overall set of estimated independent components (not only the ones to be plotted); `P_ICs` ($n_f \times n$) is the set of power spectral density of all ICs (returned by applying `pwelch` to Y'); `F` ($n_f \times 1$) is the vector of frequencies returned by `pwelch`; `chan_locs` is the name (between quote marks) of the .locs file; `Fs` is the sampling frequency; `t1` and `t2` denote the time range (in s) for plotting the IC time pattern.

e.g.

```
t1=0; % s
t2=30; % s
index_ICs=[1 2 6];
plot_IC(index_ICs,A,Y,P_ICs,F,'Standard-10-20-Caps19.locs',srate,t1,t2)
```

Function

```
Z = trapz(X,Y,DIM)
```

computes the integral of `Y` with respect to `X` using the trapezoidal method, across dimension DIM of `Y`. The length of `X` must be the same as `size(Y,DIM))`.

e.g. Suppose that vector `PSD` ($n_f \times 1$) contains the power spectral density of a single signal, computed at the frequencies `F` ($n_f \times 1$). To compute the power of the signal in the theta range (4 – 8 Hz), the following instructions can be used:

```
imin=find(F==4);
imax=find(F==8):
theta_power=trapz(F(imin:imax),PSD(imin:imax));  % 1 x 1 (scalar value)
```

In case `PSD` ($n_f \times n$) contains the power spectral density of n signals than

```
theta_power=trapz(F(imin:imax),PSD(imin:imax,:),1);  % 1 x n (row vector)
```

is a row vector containing in the i-th column the theta power of the i-th signal